DEEP LEARNING FROM FIRST PRINCIPLES | PURE NUMPY IMPLEMENTATION

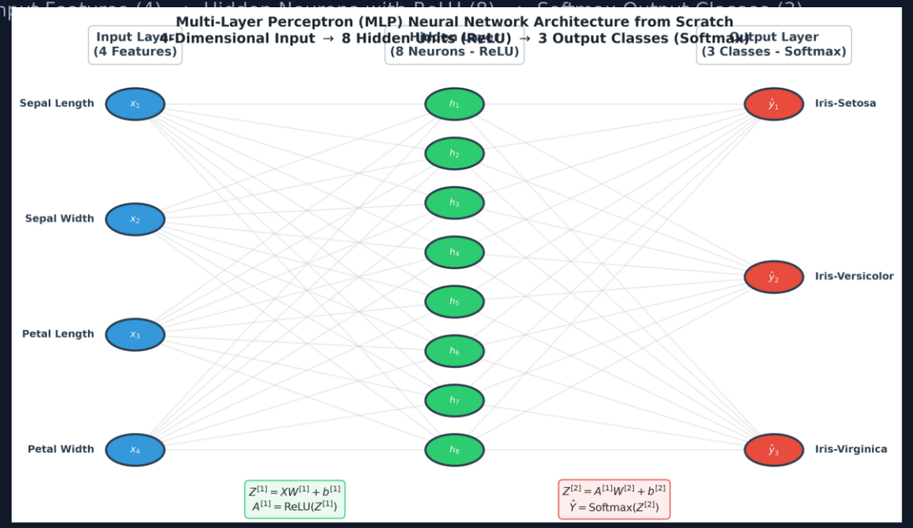
MULTI-LAYER PERCEPTRON FROM SCRATCH

Vectorized Forward Propagation, Analytical Backpropagation & Framework Benchmarking on the Iris Da

TEST ACCURACY	FRAMEWORKS	ARCHITECTURE	OPTIMIZER
96.67%	ZERO (Pure Math)	4 → 8 → 3 (MLP)	Batch GD (a=0.08)

1. NETWORK TOPOLOGY (4 → 8 → 3)

Input Features (4) → Hidden Neurons with ReLU (8) → Softmax Output Classes (3)



2. MATHEMATICAL FORMULATION & GRADIENTS

Forward propagation derivatives derived via Chain Rule without Auto-Diff frameworks

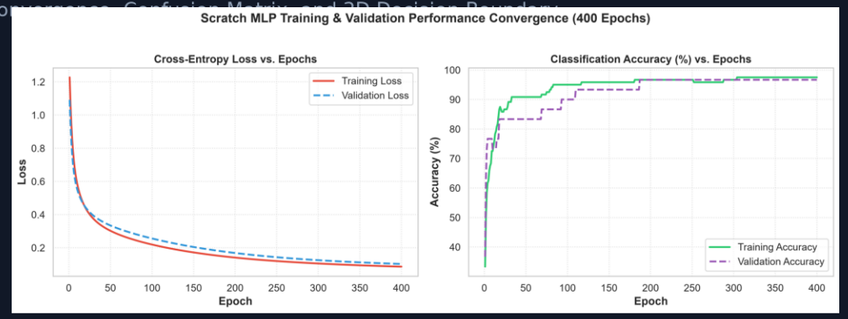
```
FORWARD PROPAGATION:
• Hidden Layer Pre-activation :   Z[1] = X · W[1] + b[1]      (W[1] ∈ R^(4×8),  b[1] ∈ R^(1×8))
• Hidden Layer Activation     :   A[1] = ReLU(Z[1]) = max(0, Z[1])
• Output Pre-activation       :   Z[2] = A[1] · W[2] + b[2]    (W[2] ∈ R^(8×3),  b[2] ∈ R^(1×3))
• Softmax Probabilities       :   Y_hat = Softmax(Z[2]) = exp(Z[2] - max(Z[2])) / Σ exp(...)

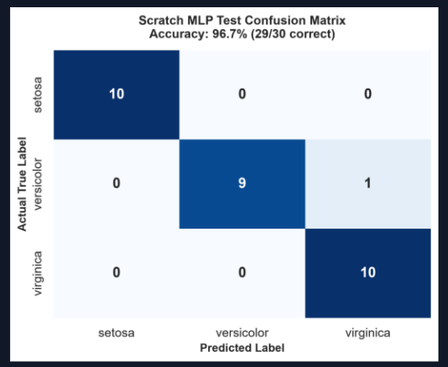
LOSS FUNCTION:
• Categorical Cross-Entropy   :   L = - (1/m) · Σ Σ [ Y_ik · log(Y_hat_ik + 1e-15) ]

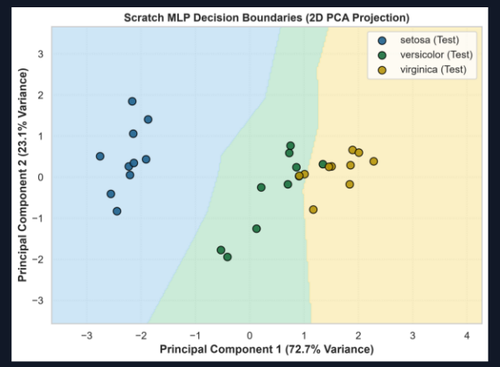
BACKPROPAGATION (CHAIN RULE DERIVATIVES):
• Output Layer Error (δ[2])   :   δ[2] = ∂L/∂Z[2] = Y_hat - Y                  (m × 3)
• Output Parameter Gradients  :   ∂L/∂W[2] = (1/m) (A[1])^T · δ[2]  |  ∂L/∂b[2] = (1/m) Σ δ[2]
• Hidden Layer Error (δ[1])   :   δ[1] = (δ[2] · (W[2])^T) ⊙ ReLU'(Z[1])        (m × 8)
• Hidden Parameter Gradients  :   ∂L/∂W[1] = (1/m) X^T · δ[1]       |  ∂L/∂b[1] = (1/m) Σ δ[1]
• Batch GD Update Rule        :   W := W - α · (∂L/∂W)   |   b := b - α · (∂L/∂b)
```

3. EXPERIMENTAL RESULTS & VISUAL DIAGNOSTICS

400 Epochs Convergence, Confusion Matrix, and 2D Decision Boundary







4. OPTIMIZATION BREAKDOWN & FRAMEWORK BENCHMARK

Key engineering steps that propelled accuracy from 63% to 96.7%

```
ACCURACY OPTIMIZATION LEVERS:
  ✓ Feature Standardization (Z-score) : Resolves non-uniform feature scales & gradient ping-pong.
  ✓ He & Xavier Normal Weight Init    : Prevents vanishing gradients across ReLU activations.
  ✓ Numerical Stability Protections   : Shift-invariance in Softmax & probability clipping in log.

FRAMEWORK BENCHMARK COMPARISON:
  • Scratch MLP (NumPy from scratch) :  96.67% Test Accuracy  |  Loss: 0.1020
  • Scikit-Learn (MLPClassifier)     :  96.67% Test Accuracy  |  Loss: 0.1035
  • TensorFlow / Keras (Dense SGD)   :  96.67% Test Accuracy  |  Loss: 0.1387
```

GITHUB REPOSITORY: https://github.com/ved-hubmax/MLP-IRIS-MODEL

TECH STACK: Python • NumPy • Scikit-Learn • TensorFlow • Matplotlib